TCP

Илья Кокорин

kokorin.ilya.1998@gmail.com

В предыдущих сериях: недостатки UDP

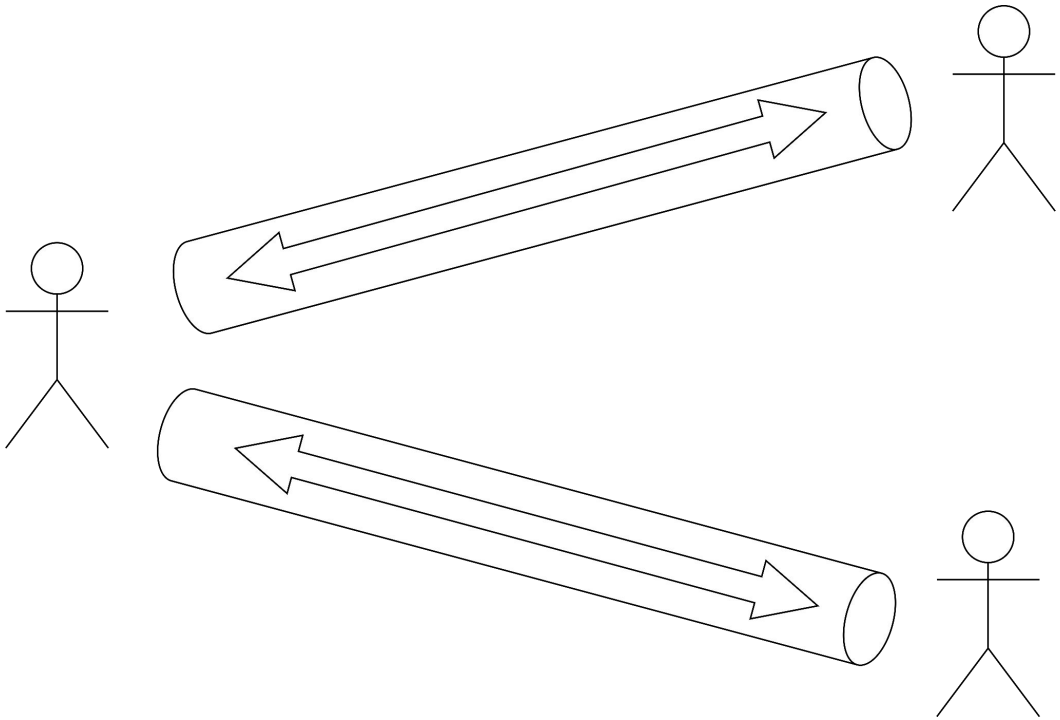
- Переупорядочивание сообщений
- Потери сообщений
- Дублирование сообщений
- И не только

TCP сокет: модель

- TCP сокет это две трубы между парой процессов
  - В первую трубу пишет первый процесс, а второй из неё читает
  - Во вторую - наоборот

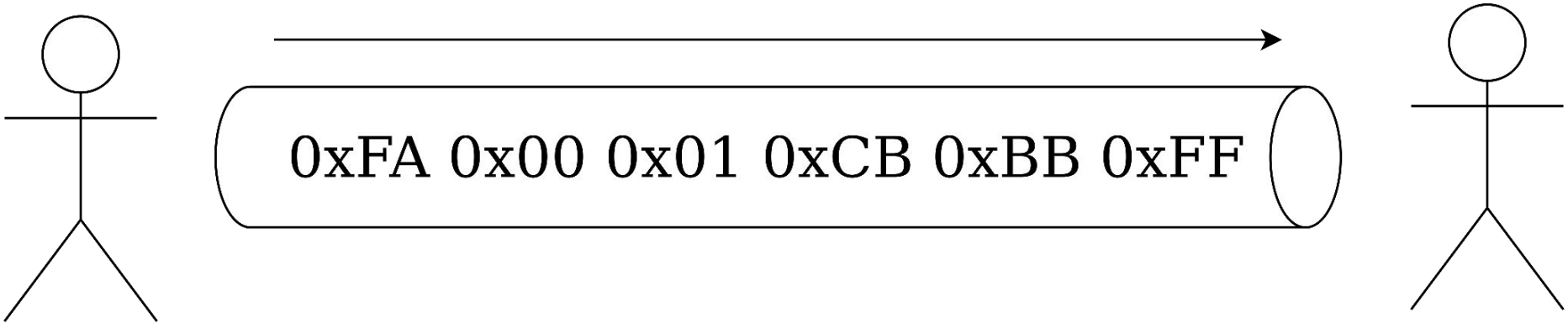
TCP: модель

- Сокет позволяет связать пару процессов
  - Для связи со вторым процессом нужен второй сокет



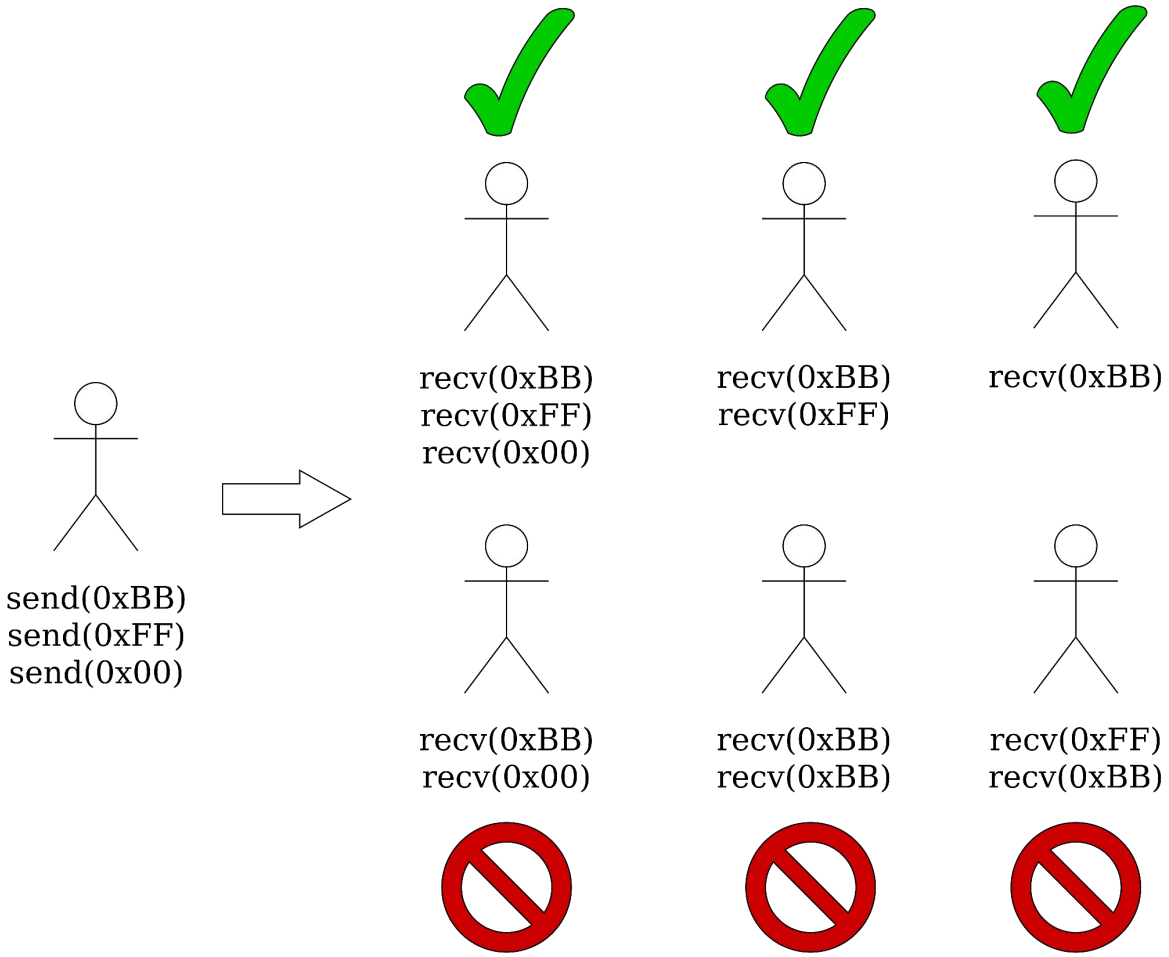
TCP: модель

- Каждая труба это непрерывный поток байт
  - Без разделения на сообщения
- Байты будут доставлены получателю в том порядке, в котором их отправил отправитель
  - Без “дырок” и дублирования



TCP: гарантии

- Байты будут доставлены получателю в том порядке, в котором их отправил отправитель
  - Без “дырок” и дублирования



TCP: непрерывность потока байт

- На стороне отправителя:
  - `write("Hello")`
  - `write("World")`
- Получателю из `read()` может вернуться:
  - `Hello`
  - `He`
  - `HelloWo`
  - `HelloWorld`
  - Любой префикс
- Границы сообщений нужно определять самому

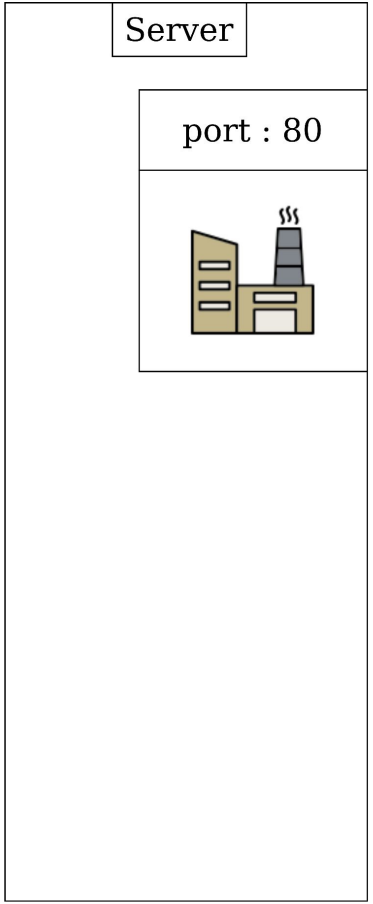
TCP: границы сообщений

- Использовать разделитель
  - Например, CRLF (`\r\n`)
  - `Message1\r\nMessage2\r\n`
- Явно задавать длину
  - Сначала пишем 4 байта - длину сообщения в байтах
  - Ну, или не четыре
  - Потом само сообщение

0x00	0x00	0x00	0x05	0x68	0x65	0x6C	0x6C	0x6F	0x00	0x00	0x00	0x03	0x6B	0x76	0x61
Length = 5				hello					Length = 3				kva		

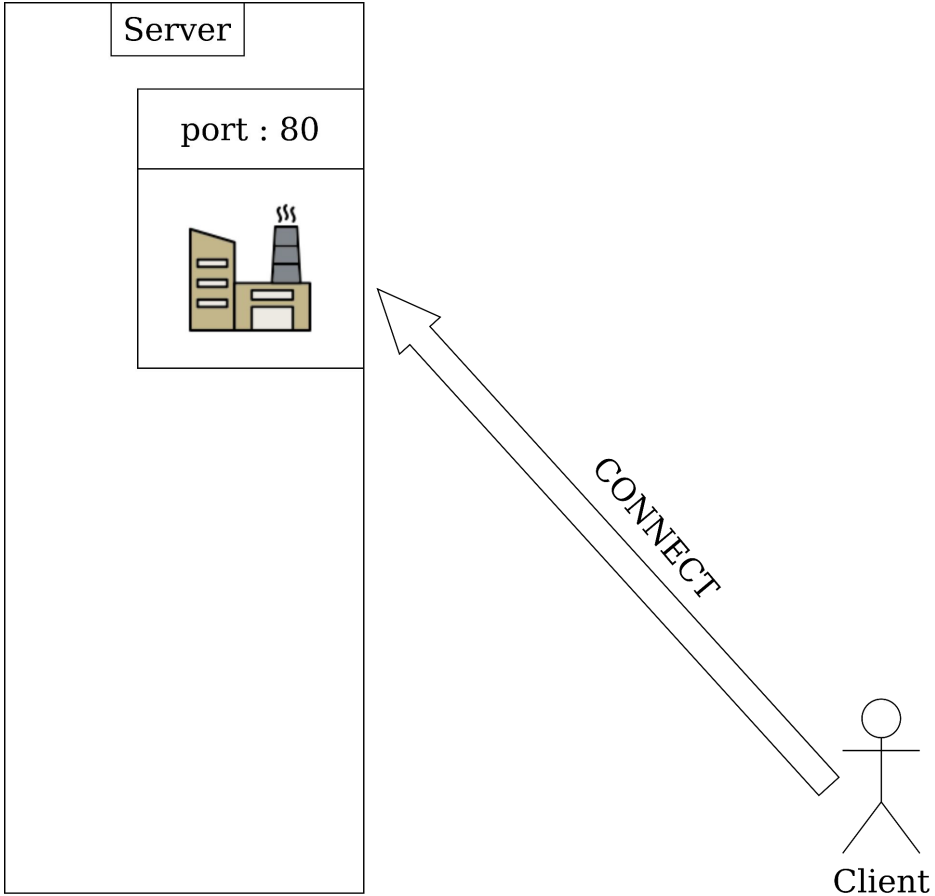
TCP сокеты: клиентские и серверные

- Серверный TCP-сокет - это трубопрокатный завод



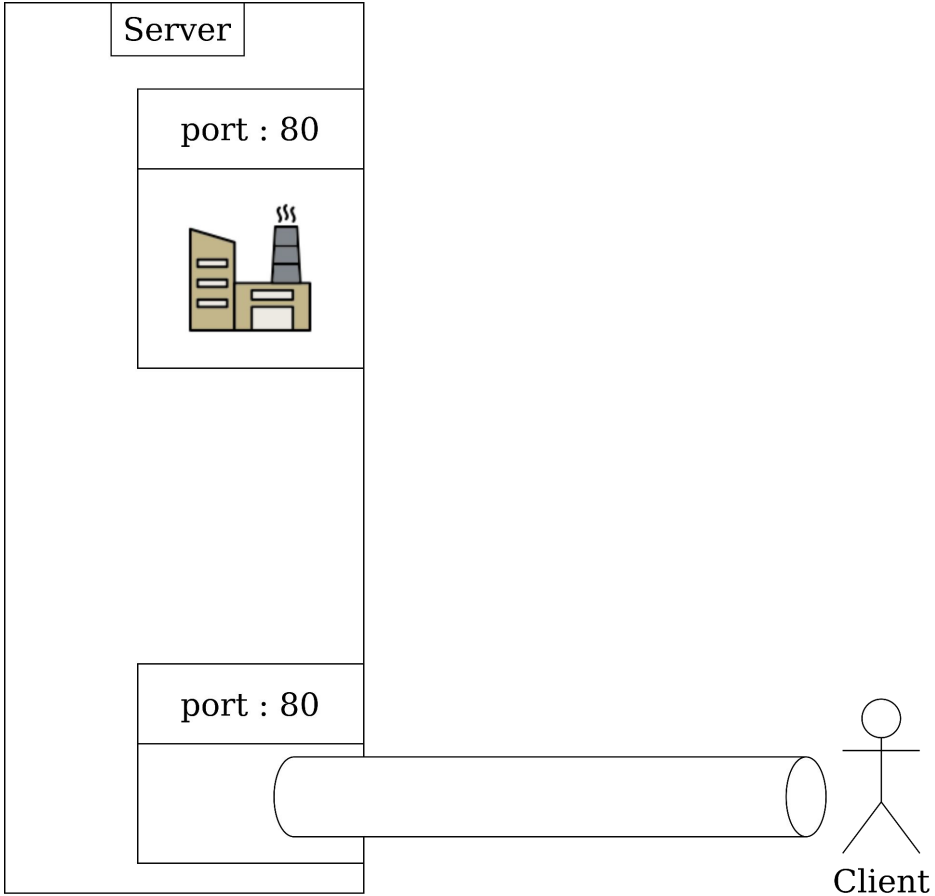
TCP сокеты: клиентские и серверные

- Клиенты приходят к нему с запросами на подключение



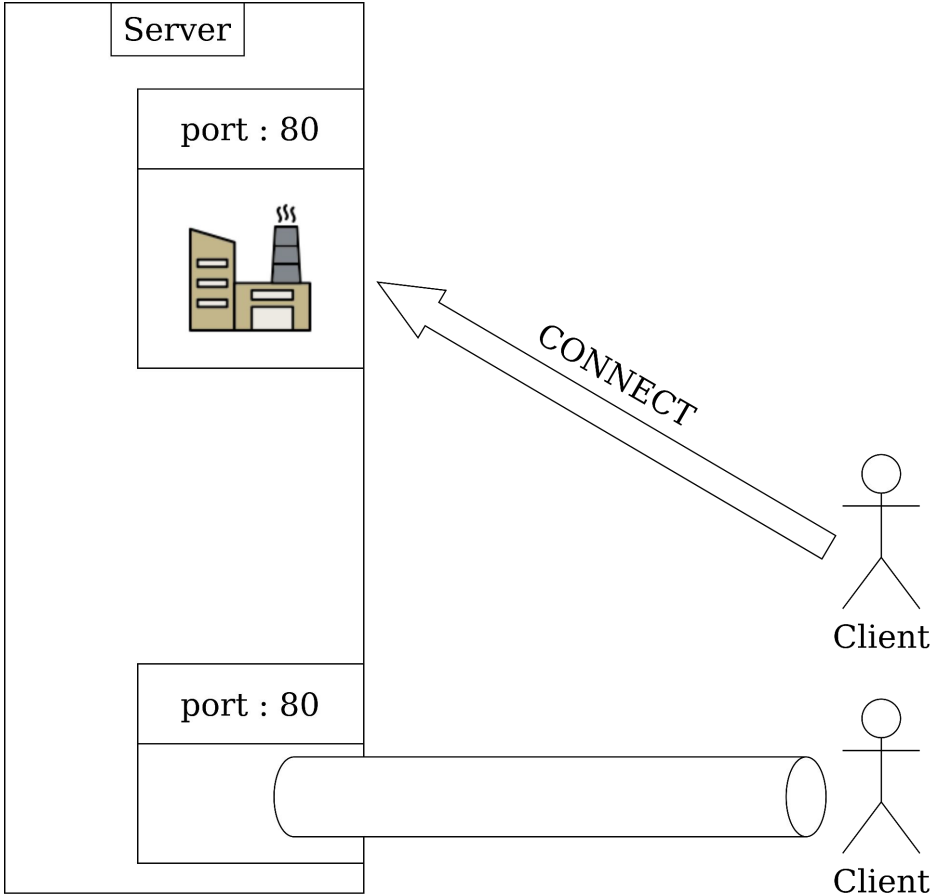
TCP сокеты: клиентские и серверные

- Серверный сокет порождает индивидуальную трубу для этого клиента
  - Порт трубы совпадает с портом серверного сокета



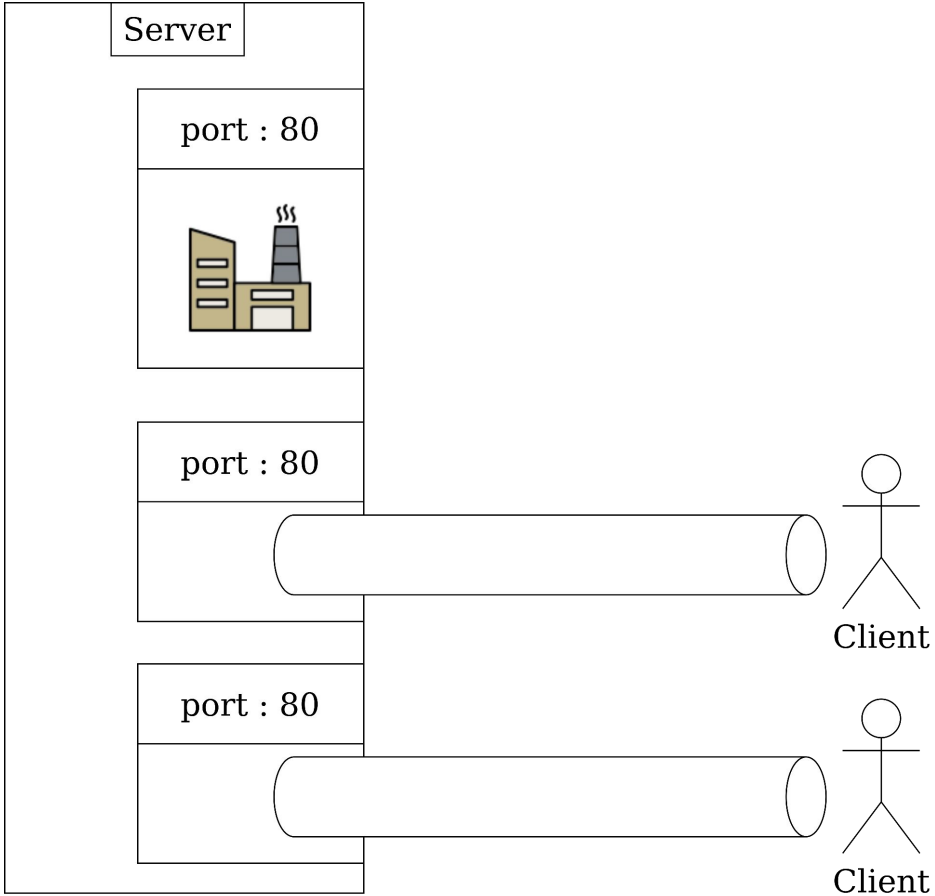
TCP сокеты: клиентские и серверные

- После этого серверный сокет может обслуживать подключения других клиентов



TCP сокеты: клиентские и серверные

- С каждым клиентом - своя труба
  - У всех труб одинаковые порты

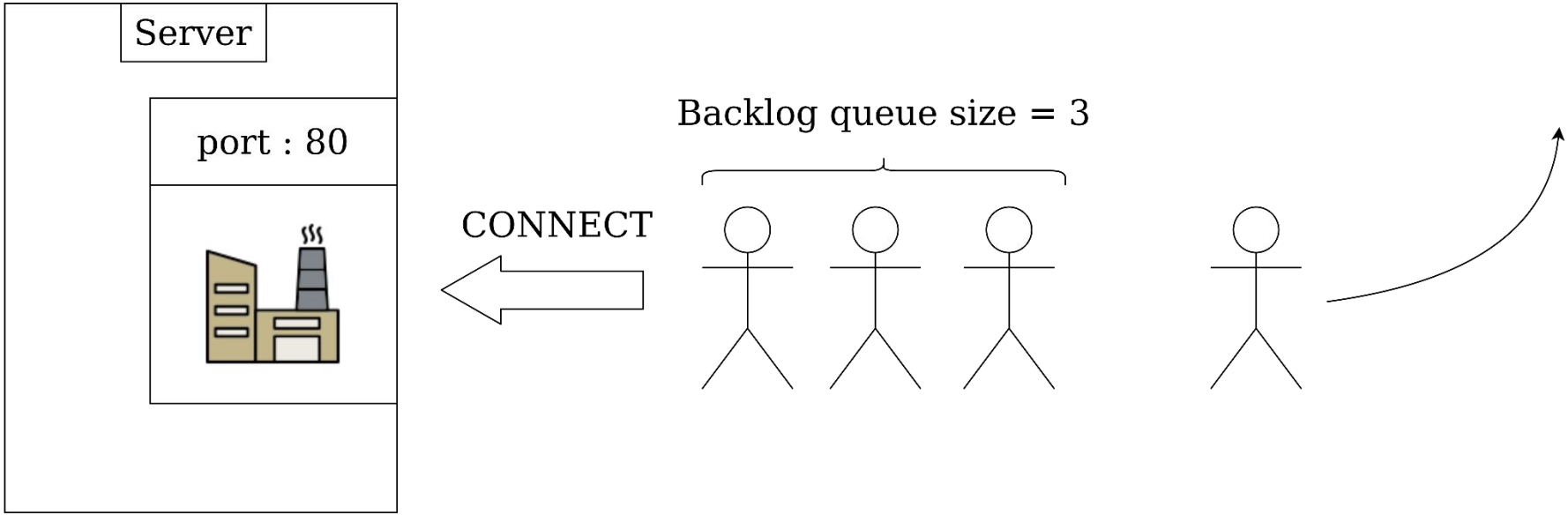


TCP: операции с серверным сокетом

- Создание
  - `fd := socket(IPPROTO_TCP)`
- Привязка к адресу
  - `bind(fd, host, port)`
- Перевод в серверный режим
  - `listen(fd, bqs)`
- Получение клиентского соединения
  - `conn := accept(fd)`
  - Поток получит управление когда придёт клиент
- Работа с соединением
- Закрытие соединения
  - `close(fd)`

TCP: Backlog queue

- Пришёл новый клиент
- Сервер прямо сейчас не ждёт в `accept()`
- ОС ставит клиента в очередь
  - Или сообщает о том, что очередь переполнена



TCP: операции с клиентским сокетом

- Создание
  - `fd := socket(IPPROTO_TCP)`
- Соединение с сервером
  - `connect(fd, server_host, server_port)`
  - Клиентский порт будет выбран **случайно**
  - Возможен *port exhaustion*
  - Иначе надо сделать
    `bind(fd, local_host, local_port)`
- Работа с соединением
- Закрытие
  - `close(fd)`

TCP: операции с соединением

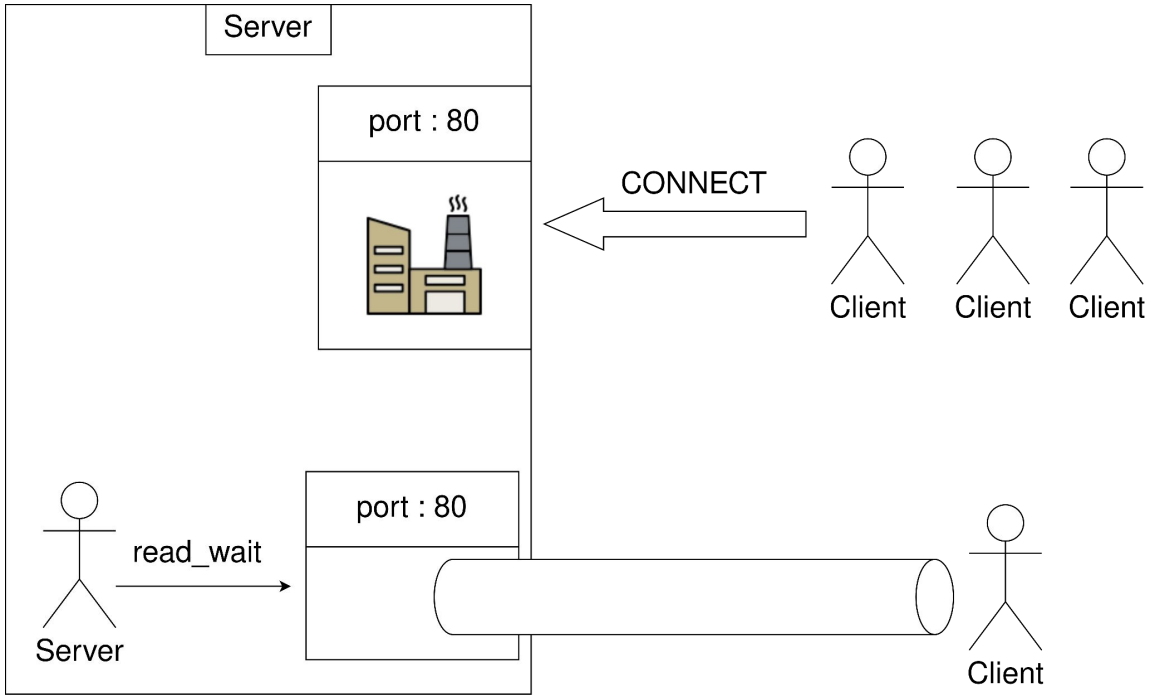
- Чтение
  - `bytes := read(conn)`
  - Читает байты, отправленные другой стороной
  - Блокирующий вызов
- Запись
  - `written := write(conn, bytes, offset)`
  - Записывает байты, чтобы другая сторона могла их прочитать
  - Блокирующий вызов
- Стороны симметричны
- Закрытие
  - `close(conn)`

TCP: операции с закрытым соединением

- Соединение закрывается с противоположной стороны
  - Мы пока об этом не знаем и продолжаем с ним работать
- `bytes := read(conn)`
  - Получаем массив размера 0
- `res := write(conn, bytes)`
  - Возвращает `-1`
  - `errno = EPIPE`

TCP: ожидающие клиенты

- Сервер ждёт в системном вызове `read(conn)`
- Другие клиенты пришли и ждут
- Но `new_conn := accept(fd)` никто не вызывает
- Новые клиенты ждут, пока существующий клиент пришлёт ещё данных

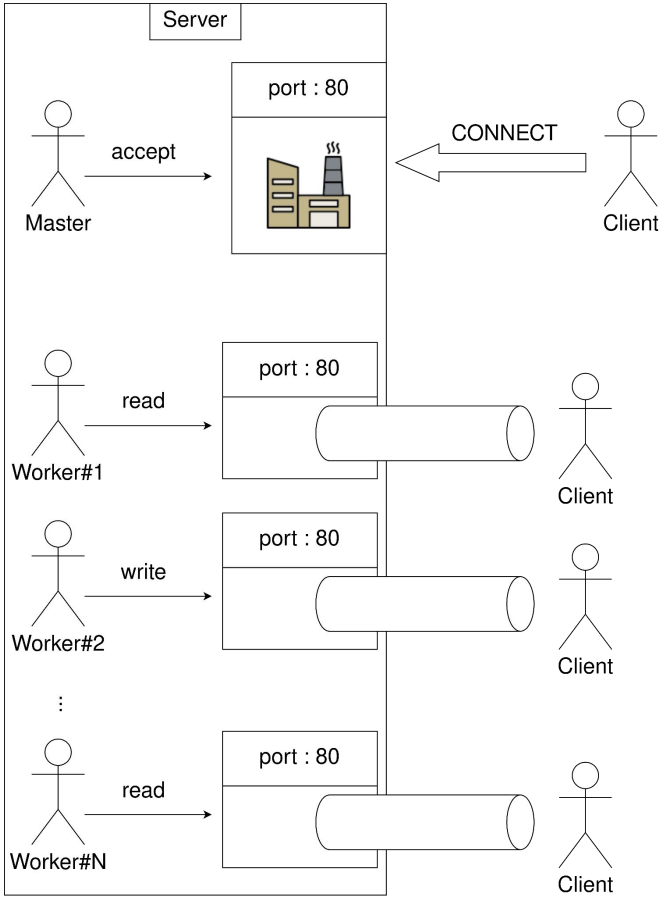


TCP: многопоточный сервер

- Обрабатываем каждого клиента в отдельном потоке

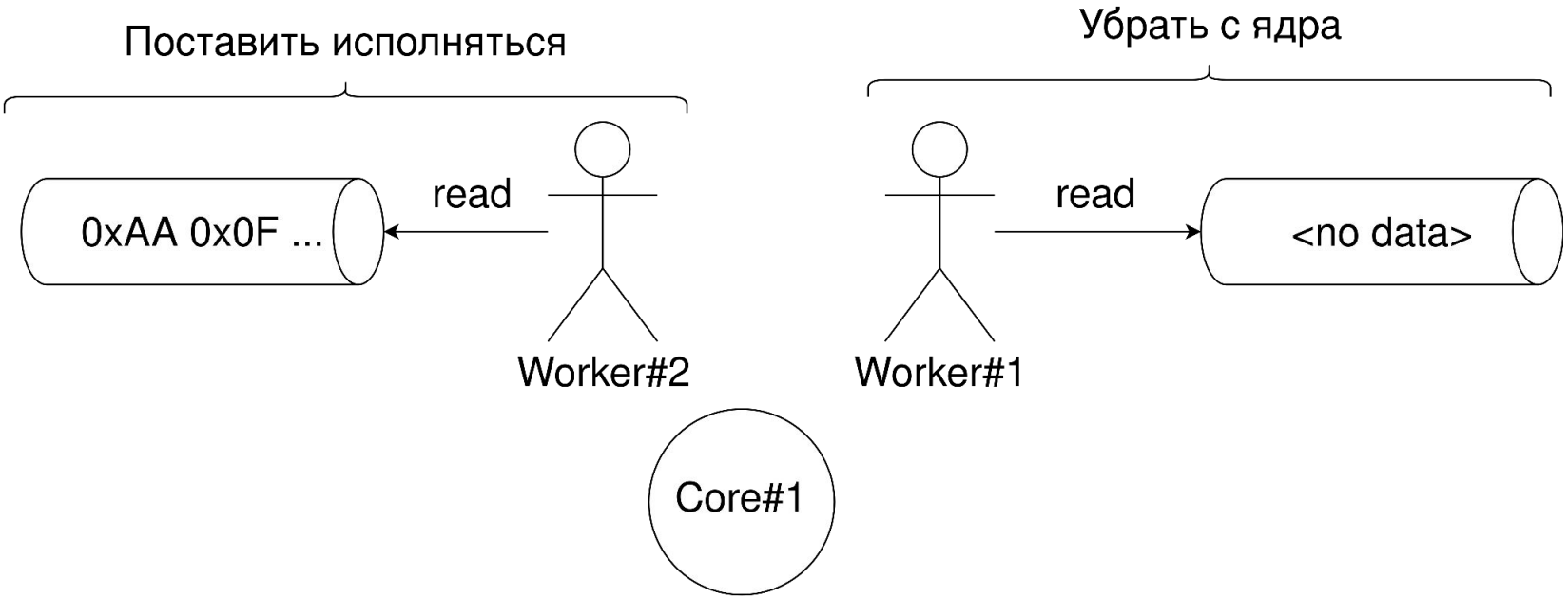
```
fd = socket(IPPROTO_TCP)
bind(fd, host, port)
listen(fd, bqs)
while true:
    conn := accept(fd)
    thread {
        process_client(conn)
        close(conn)
    }
close(fd)
```

TCP: многопоточный сервер

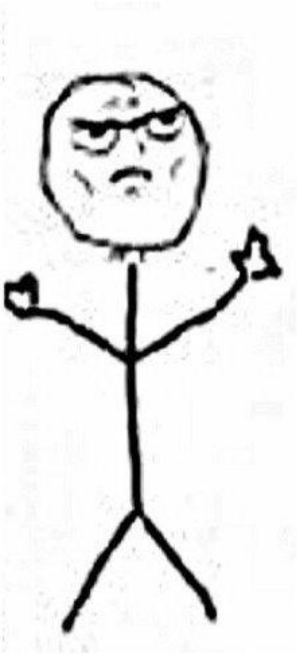


Цена потока

- Поток занимает оперативную память
- context switch не бесплатные
- Сложность обновления общего состояния



Что почитать & посмотреть

- Ephemeral port exhaustion and how to avoid it
- How to stop running out of ephemeral ports and start to love long-lived connections
- *Александр Тоболь.* Пишем свой протокол поверх UDP
- *Александр Тоболь.* UDP против TCP, или Будущее сетевого стека
- The 10K problem
- Measuring context switching and memory overheads for Linux threads
- *Roman Elizarov.* Introduction to coroutines

спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание